

Linguagem C – Funções e procedimentos I

Este texto foi adaptado pelos professores Alexandre Rômolo Moreira Feitosa e Francisco Pereira Junior (Thesko).

Funções

Funções (também chamadas de **rotinas**, ou **subprogramas**) são a essência da programação estruturada. Funções são segmentos de programa que executam uma determinada tarefa específica. Já vimos o uso de funções providenciadas pelas bibliotecas-padrão da linguagem C nos capítulos anteriores, como: `sqrt()`, `toupper()`, `getch()` ou `putchar()`.

Além de usar funções pré-definidas, o programador pode escrever suas próprias rotinas. São as chamadas de **funções de usuário** ou rotinas de usuário. Deste modo pode-se segmentar um programa grande em vários programas menores. Esta segmentação é chamada de **modularização** e permite que cada segmento seja escrito, testado e revisado individualmente sem alterar o funcionamento do programa como um todo. Permite ainda que um programa seja escrito por vários programadores ao mesmo tempo, cada um escrevendo um segmento separado. Neste capítulo, veremos como escrever funções de usuário na linguagem C.

Estrutura das funções de usuário

A estrutura de uma função de usuário é muito semelhante a estrutura dos programas que escrevemos até agora. Uma função de usuário constitui-se de um **bloco de instruções** que definem os procedimentos efetuados pela função, um **nome** pelo qual a chamaremos e uma **lista de argumentos** passados a função (conforme necessidade). Chamamos este conjunto de elementos de **definição da função**.

Exemplo: o código mostrado abaixo é uma função definida pelo usuário para calcular a média aritmética de dois números reais:

```
float minhaMedia(float a, float b){
    float med;
    med = (a + b) / 2.0;
    return(med);
}
```

Neste exemplo definimos uma função chamada `minhaMedia` que recebe dois argumentos do tipo `float`: `a` e `b`. A média destes dois valores é calculada e armazenada na variável local `med`, declarada internamente na função. Essa função ainda retorna, para o programa que a chamou, um valor também do tipo `float`, que neste caso é o valor da variável `med`. Este retorno de valor é feito pela função `return()` que termina a execução da função e devolve o valor de `med` para o programa que a chamou.

Depois de definirmos uma função, podemos usá-la dentro de um programa qualquer e assim dizemos que estamos fazendo uma **chamada** à função.

Exemplo: No exemplo abaixo chamamos a função `minhaMedia(...)` dentro de um programa principal:

```
void main() {
    float num_1, num_2, med;
    puts("Digite dois números:");
    scanf("%f %f", &num_1, &num_2);
    med = minhaMedia(num_1, num_2);    // chamada à função minhaMedia
    printf("\nA media destes números é %f", med);
}
```

Definição de funções

De modo formal, a sintaxe de uma função é a seguinte:

```
<tipo_de_retorno> <nome_da_função>(<tipo_1 arg_1, tipo_2 arg_2, ...>{
    [bloco de instruções da função]
}
```

A primeira linha da função contém a **declaração** da função que é definida pelo `nome_da_função`, pelo `tipo_de_retorno` e pela `lista_de_argumentos` que eventualmente ela receberá. Em seguida, dentro de chaves `{...}`, definimos o bloco de instruções que esta função irá executar.

O `tipo_de_retorno` da função especifica qual o tipo de dado será devolvido pela função, podendo ser qualquer tipo de dado válido (`char`, `int`, `float`, e `double`). Se a função não retorna nenhum valor para o programa que a chamou devemos definir o retorno como `void`, ou seja, um retorno ausente. Se nenhum tipo de retorno for explicitamente especificado, o compilador entenderá que o retorno será do tipo `int`. Vale notar que existe apenas **um** valor de retorno para as funções em linguagem C. **Não** podemos fazer o retorno de **dois ou mais** valores como em algumas linguagens (no MatLab: `[media, desvio] = estat(a, b, c, d, e)`). Porém isto não é uma limitação séria, pois o uso de ponteiros, que veremos adiante, contornará esse “problema”.

Por ser um identificador, o `nome_da_função` segue as mesmas regras de definição de identificadores. A `lista_de_argumentos` de uma função especifica quais são os valores que a função vai receber quando for chamada. As variáveis da lista de argumentos são manipuladas normalmente no corpo da função, como as demais variáveis declaradas localmente.

A execução de uma função termina em um dos dois momentos, o que ocorrer primeiro:

- a) na chave de fechamento da função `“}”`, ou
- b) na instrução `return`.

Em ambos os casos o controle do fluxo de execução do programa é transferido ao programa chamador da função. Entretanto, no segundo, a instrução `return` tem duas finalidades: devolver um **valor** calculado pela função e determinar o **fim lógico** da rotina.

Como podemos observar, uma função criada pelo usuário pode ou não retornar um valor e pode ou não receber argumentos, quando for chamada. Desta forma, resumidamente, temos 4 combinações possíveis que podem ser utilizadas na criação de uma função de usuário. Uma delas deverá ser escolhida conforme a necessidade da sua regra de negócio:

- a) Função **sem** retorno e **sem** parâmetro;
- b) Função **com** retorno e **sem** parâmetro;
- c) Função **sem** retorno e **com** parâmetro;
- d) Função **com** retorno e **com** parâmetro.

a) Exemplo de função **SEM** retorno e **SEM** parâmetro e seu uso.

<pre>void minhaMedia(void){ float num1 = 3.45, num2 = 96.55, med = 0; med = (num1 + num2) / 2.0; printf("Media = %f", med); }</pre>	<pre>void main(){ ... minhaMedia(); }</pre>
---	---

b) Exemplo de função **COM** retorno e **SEM** parâmetro e seu uso.

<pre>float minhaMedia(void){ float num1 = 3.45, num2 = 96.55, med = 0; med = (num1 + num2) / 2.0; return med; }</pre>	<pre>void main(){ float var = 0; ... var = minhaMedia(); printf("Media = %f", var); }</pre>
---	---

c) Exemplo de função **SEM** retorno e **COM** parâmetro e seu uso.

<pre>void minhaMedia(float a, float b){ float med = 0; med = (a + b) / 2.0; printf("Media = %f", med); }</pre>	<pre>void main(){ float num1 = 3.45, num2 = 96.55, ... minhaMedia(num1, num2); }</pre>
--	--

d) Exemplo de função **COM** retorno e **COM** parâmetro e seu uso.

<pre>float minhaMedia(float a, float b){ float med; med = (a + b) / 2.0; return(med); }</pre>	<pre>void main(){ float num1 = 3.45, num2 = 96.55, var = 0; ... var = minhaMedia(num1, num2); printf("Media = %f", var); }</pre>
---	--

Localização das funções

Existem basicamente duas posições possíveis para escrevermos o corpo de uma função: ou **antes**, ou **depois** do programa principal. Podemos ainda escrever uma função no **mesmo arquivo** do programa principal ou em **arquivo separado**.

Corpo da função antes do programa principal (no mesmo arquivo)

Quando escrevemos a definição de uma função **antes** do programa principal e no mesmo arquivo deste, nenhuma outra instrução é necessária. A sintaxe geral para isto é a seguinte:

Sintaxe: Uma função escrita antes do programa principal:

```
tipo nomef(...){                // definição da função
    [corpo de função]
}
void main(){                    // programa principal
    ...
    var = nomef(...);           // chamada da função
    ...
}
```

Exemplo: Função `minhaMedia()` escrita antes do programa principal.

```
float minhaMedia(float a, float b){ // função
    float med;
    med = (a + b) / 2.0;
    return(med);
}
void main(){                      // programa principal
    float num_1, num_2, med;
    puts("Digite dois números:");
    scanf("%f %f", &num_1, &num_2);
    med = minhaMedia(num_1, num_2); // chamada da função
    printf("\nA media destes números é %f", med);
}
```

Corpo da função depois do programa principal (no mesmo arquivo)

Quando escrevemos a definição de uma função **depois** do programa principal e no mesmo arquivo deste, devemos incluir um **protótipo** da função. Um protótipo é uma instrução que define o nome da função, seu tipo de retorno e a quantidade e o tipo dos argumentos desta função. Serve também como uma declaração da existência dessa função para o compilador, que conhecerá de antemão, quais são as funções usadas no programa principal e quais os seus tipos. A sintaxe geral para isto é a seguinte:

Sintaxe: Uma função escrita depois do programa principal:

```
tipo nomef(...);           // protótipo da função
void main(){               // programa principal
    ...
    var = nomef(...);      // chamada a função
    ...
}
tipo nomef(...){           // definição da função
    [corpo de função]
}
```

Exemplo: Função `minhaMedia()` escrita depois do programa principal.

```
float minhaMedia(float a, float b); // protótipo de minhaMedia()
void main(){                        // programa principal
    float num_1, num_2, med;
    puts("Digite dois números:");
    scanf("%f %f", &num_1, &num_2);
    med = minhaMedia(num_1, num_2); // chamada a função
    printf("\nA media destes números é %f", med);
}
float minhaMedia(float a, float b){ // função minhaMedia()
    float med;
    med = (a + b) / 2.0;
    return(med);
}
```

Observe que o protótipo de uma função nada mais é do que a **declaração** da função sem o seu corpo (cabeçalho). Observe ainda que o protótipo da função é terminado por um (;) e na lista de argumentos do protótipo pode ser escritos apenas os **tipos** dos argumentos, sem o nome das variáveis. Por exemplo:

```
float minhaMedia(float, float);
```

Corpo da função escrito em arquivo separado

Em linguagem C, como em muitas outras linguagens, é permitido que o usuário crie uma função em **um arquivo** e um programa que a chame em outro **arquivo distinto**. Esta facilidade permite a criação de **bibliotecas de usuário**: um conjunto de arquivos contendo funções escritas pelo usuário. Esta possibilidade é uma grande vantagem, muito utilizada por programadores profissionais.

Quando escrevemos a definição de uma função em **arquivo separado** do programa principal devemos **incluir** este arquivo no conjunto de arquivos de **compilação** do programa principal. Esta inclusão é feita com a diretiva `#include`. Esta diretiva instrui o compilador para incluir na compilação do programa outros arquivos que contém a definição das funções de usuário e de biblioteca.

Sintaxe: A sintaxe de inclusão de funções de usuário é a seguinte:

```
#include "path"           // inclusão da função
void main() {             // programa principal
    ...
    var = nomef(...);     // chamada a função
    ...
}
```

Na diretiva `#include`, indicamos entre aspas duplas o caminho de localização do arquivo onde está definida a função chamada.

Exemplo: A função `minhaMedia()` está escrita em um arquivo separado.

```
#include "c:\tc\userbib\stat.h" // inclusão da função
void main() {                   // programa principal
    float num_1, num_2, med;
    puts("Digite dois números:");
    scanf("%f %f", &num_1, &num_2);
    med = minhaMedia(num_1, num_2); // chamada a função
    printf("\nA media destes números é %f", med);
}
```

Observação: Um arquivo pode conter a definição de uma ou mais funções. Em geral, quando o arquivo possui apenas uma função ele é nomeado com o mesmo nome da função e extensão `*.cpp` ou `*.c`. Por exemplo, poderíamos definir a função `minhaMedia()` no arquivo `minhaMedia.c`. Quando um arquivo possui a definição de mais de uma função, ele é nomeado com a extensão `*.h` ou `*.lib`. Por exemplo: poderíamos criar um conjunto de funções estatísticas chamadas `media()`, `dsvpd()`, `moda()`, `max()`, `min()`, etc. definindo-as em um arquivo chamado `stat.h`.

Regra de escopo para variáveis

A regra de escopo define o **âmbito de validade** de variáveis. Em outras palavras define onde as variáveis e funções são reconhecidas. Em linguagem C, uma variável só pode ser **usada** após ser **declarada**. Isto por que o processador deve reservar um local da memória para armazenar os valores atribuídos à variável. Porém, o local do programa, onde uma variável é declarada define ainda seu **escopo** de validade. Uma variável pode ser **local**, **global** ou **formal** de acordo com o local de declaração.

Variáveis Locais: uma variável é dita *local*, se for declarada **dentro do bloco** de uma função. Uma variável local tem validade apenas dentro do bloco onde é declarada, isto significa que podem ser apenas acessadas e modificadas dentro desse bloco. O espaço de memória alocado para esta variável é **criado** quando a execução do bloco é iniciada e o espaço é **destruído** quando o bloco é encerrado, assim variáveis de mesmo nome mas declaradas em blocos distintos, são para todos os efeitos, variáveis distintas.

Exemplo:

```
float minhaMedia(float a, float b){
    float med;
    med = (a + b) / 2.0;
    return(med);
}
```

```

void main(){
    float num_1, num_2, med;
    puts("Digite dois números:");
    scanf("%f %f", &num_1, &num_2);
    med = minhaMedia(num_1, num_2);
    printf("\nA media destes números é %f", med);
}

```

No exemplo acima, `med` é uma variável local definida pela função `minhaMedia()`. Outra variável `med` é também definida pela função `main()`. Para todos os efeitos estas variáveis são distintas.

Variáveis Formais: uma *variável formal* é uma variável local declarada na **lista de parâmetros** de uma função. Deste modo, tem validade apenas dentro da função onde é declarada, porém serve de suporte para os valores passados pelas funções. As variáveis formais na **declaração** da função e na **chamada** da função podem ter nomes distintos. A única exigência é de que sejam do mesmo tipo.

Por serem variáveis locais, os valores que uma função passa para outra não são alterados pela função chamada. Este tipo de passagem de argumentos é chamado de **passagem por valor**, pois os valores das variáveis do programa chamador são copiados para as correspondentes variáveis da função chamada.

No exemplo acima, `a` e `b` são parâmetros formais declarados na função `minhaMedia()`. Observe que a função é chamada com os valores de `num_1` e `num_2`. Mesmo que os valores de `a` e `b` fossem alterados os valores de `num_1` e `num_2` não seriam modificados.

Variáveis Globais: uma variável é dita *global*, se for declarada **fora do bloco** de uma função. Uma variável global tem validade no escopo de todas as funções, isto é, pode ser acessada e modificada por qualquer função. O espaço de memória alocado para esta variável é **criado** no momento de sua declaração e **destruído** apenas quando o programa é encerrado, ou seja, fica ocupando memória em todo o tempo de execução do programa.

Exemplo: Uso de variáveis globais.

```

float a, b, med;                // declaração de variáveis globais

void minhaMedia(void){
    med = (a + b) / 2.0;
}

void main(){
    puts("Digite dois números:");
    scanf("%f %f", &a, &b);
    minhaMedia();
    printf("\nA media destes números é %f", med);
}

```

No exemplo anterior, `a`, `b`, `med` são variáveis globais, definidas fora dos blocos das funções `minhaMedia()` e `main()`. Deste modo, ambas as funções têm pleno acesso às variáveis, podendo ser acessadas e modificadas por qualquer uma. Assim, não é necessária a passagem de parâmetros para a função.

Passando Vetores para Funções

Vetores, assim como variáveis, podem ser usados como argumentos de funções. Vejamos como se declara uma função que recebe um vetor e como se passa um vetor para uma função.

Sintaxe: Na *passagem* de vetores para funções usamos a seguinte sintaxe:

```
nome_da_função(nome_do_vetor);
```

onde: *nome_da_função* é o nome da função que se está chamando.

nome_do_vetor é o nome do vetor que queremos passar. Indicamos **apenas** o nome do vetor, **sem** colchetes e sem índices. Se indicarmos um índice estamos passando **um elemento** do vetor e não mais o vetor como um todo.

Sintaxe: Na *declaração* de funções que recebem vetores:

```
<tipo_de_retorno> <nome_da_função>(<tipo_dado_vetor nome_vetor[]){  
    ...  
}
```

onde: *tipo_de_retorno* é o tipo de retorno da função.

nome_da_função é o nome da função.

tipo_dado_vetor é o tipo de dado dos elementos do vetor.

nome_vetor é o nome do vetor.

Observe que depois do nome do vetor temos um índice vazio `[]` para indicar que estamos recebendo um vetor.

Exemplo: Observe o exemplo abaixo:

Na declaração da função:

```
float media(float vetor[], float n){           //declaração da função  
    ...  
}
```

Na chamada da função:

```
#define MAX 10  
void main(){  
    float n = 10;  
    float valor[MAX];           // declaração do vetor  
    ...  
    med = media(valor, n);      // passagem do vetor para a função  
    ...  
}
```

Atenção: Ao contrário das variáveis comuns, o conteúdo de um vetor **pode ser modificado** pela função chamada. Isto significa que podemos passar um vetor para uma função e alterar os valores de seus elementos. Isto ocorre porque a passagem de **vetores** para funções é feita de modo especial dito **passagem por endereço**. Portanto, devemos ter cuidado ao manipular os elementos de um vetor dentro de uma função para não modificá-los por descuido.

Passando Vetores Multidimensionais (matrizes) para Funções

A sintaxe para *passagem* de vetores multidimensionais para funções é semelhante à passagem de vetores unidimensionais: chamamos a função e passamos o **nome** do vetor, **sem** colchetes e sem índices. A única mudança ocorre na declaração de funções que recebem vetores multidimensionais:

Sintaxe: Na *declaração* de funções que recebem matrizes:

```
<tipo_de_retorno> <nome_da_função>(<tipo_dado nome_matriz[][]...[]){  
    ...  
}
```

Observe que depois do nome do `matriz` temos os colchetes indicando quantas dimensões existem na matriz. O mais comum é usar matriz com 2 dimensões.

Exemplo: Observe o exemplo abaixo:

Na declaração da função:

```
int max(int vetor[][]){           // declaração da função  
    ...  
}
```

Na chamada da função:

```
void main(){  
    int valor[5][7];             // declaração da matriz  
    ...  
    med = max(valor);           // passagem da matriz para a função max  
    ...  
}
```

Observações: Algumas observações a respeito de vetores multidimensionais (matriz) podem ser feitas:

- Do mesmo modo que vetores unidimensionais, os vetores multidimensionais podem ter seus elementos modificados pela função chamada.
- Os índices dos vetores multidimensionais, também começam em 0. Por exemplo: `vet[0][0]`, é o primeiro elemento da matriz.

Embora uma tabela não seja a única maneira de visualizar um vetor bidimensional (matriz de 2 dimensões), podemos entender o **primeiro** índice do vetor como o índice de **linhas** da tabela e o **segundo** índice do vetor como índice das **colunas**.